



# CodeType Trainer

A Real-Time Syntax Typing Tutor Powered by LLMs

Final Project | Jon Steen | Graduate Course CS600



# The Problem: Syntax Fluency

- **</> Muscle Memory Gap:** Standard typing tutors use English prose, not code syntax like `{}`, `[]`, or `⇒`.
- **</> Language Specificity:** Developers need to practice the idiomatic patterns of specific languages (Python vs. JS).
- **</> Speed vs. Accuracy:** In live coding interviews or rapid development, syntax errors break flow.
- **</> Goal:** Build a tool that generates infinite, valid coding scenarios for practice.





# The Solution: CodeType Trainer

---



## AI-Generated Content

Leverages a fine-tuned GPT-3.5 model to generate randomized, syntactically correct code snippets on demand. No static databases.



## Real-Time Metrics

Tracks WPM, accuracy, and error count character-by-character using a custom JavaScript implementation embedded in Streamlit.



# System Architecture

- </> Frontend:** Streamlit (Python) handles state management and UI layout.
- </> Backend Logic:** OpenAI API (`ft:gpt-3.5-turbo`) generates content dynamically.
- </> Bridge:** `streamlit.components.v1` renders a custom HTML/JS isolated iframe.
- </> Data Flow:** Python fetches snippet → Sends to JS → JS captures events → Returns metrics to Python.



# Technology Stack

---



## Python & Streamlit

Core application logic, session state management (`st.session_state`), and UI framework.



## OpenAI API

Fine-tuned model `ft:gpt-3.5-turbo-0125` for strict output formatting and difficulty scaling.



## HTML5 / JS

Custom component for low-latency event handling (keydown listeners) and visual feedback.



# The Fine-Tuned Model

Why fine-tune `gpt-3.5-turbo`?

- **</> Strict Formatting:** The model is trained to return raw code *only* (no markdown, no chatty explanations).
- **</> Difficulty Tiers:** Custom logic for `BEGINNER`, `INTERMEDIATE`, and `ADVANCED` complexity.
- **</> Negative Constraints:** Trained to avoid repeating patterns from previous snippets context.
- **</> Efficiency:** Smaller prompts and faster completion times compared to few-shot prompting.





# Key Implementation Details

---

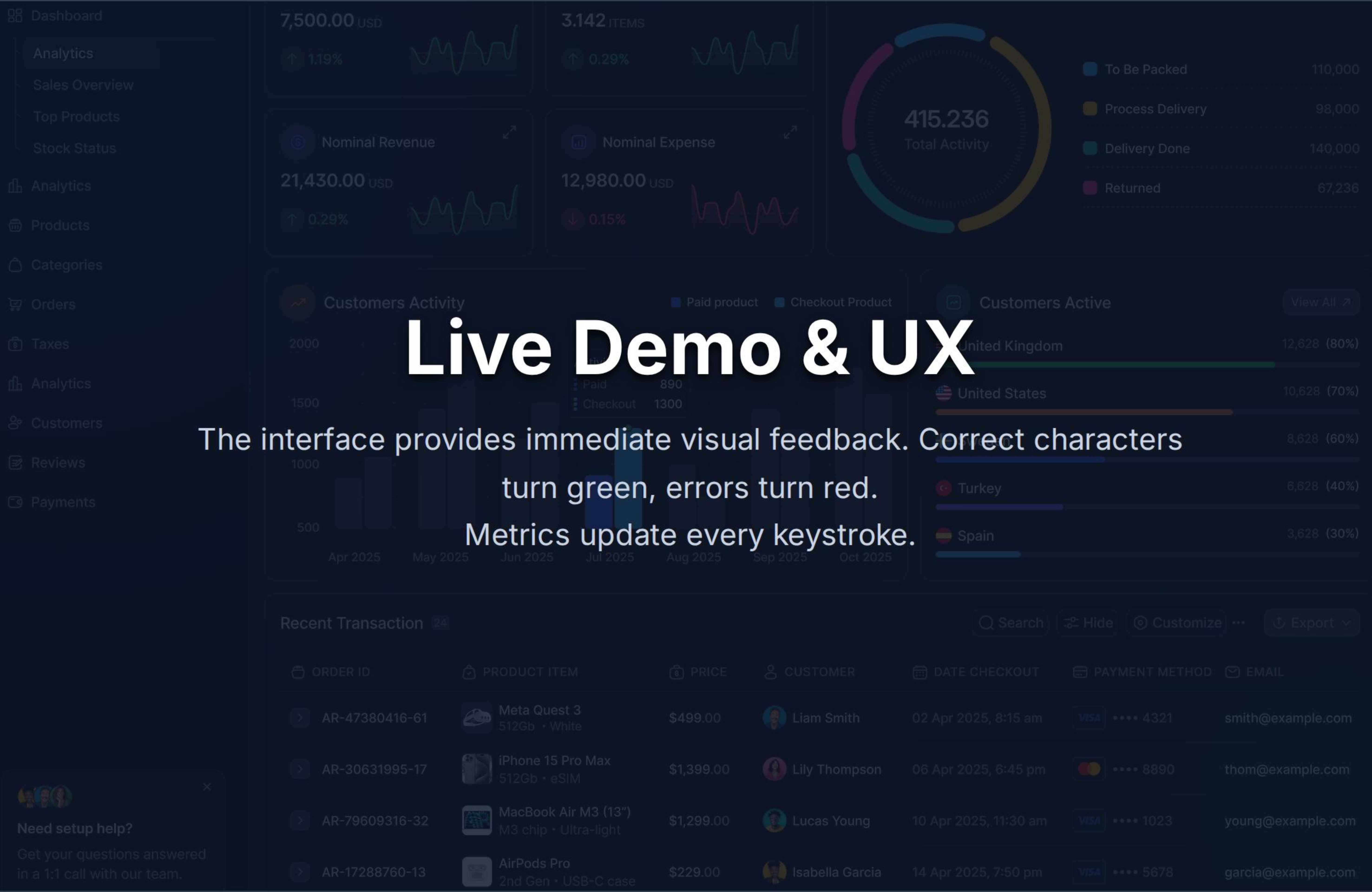
## State Management

Used `st.session_state` to persist the current snippet, user stats, and history across re-runs. This prevents the snippet from refreshing whenever a user interacts with a widget.

## The Custom Component

Streamlit's default text area doesn't support real-time color highlighting. I injected a custom HTML/CSS block using `components.html` to create a reactive typing surface that calculates WPM in the client browser.





# Live Demo & UX

The interface provides immediate visual feedback. Correct characters turn green, errors turn red.

Metrics update every keystroke.



# Development Challenges

---



## Prompt Leakage

Initially, the model would wrap code in markdown blocks. Fine-tuning datasets had to be scrubbed to enforce "raw code only" output.



## Latency

Generating code on the fly takes time. Solution: Caching strategies and optimized token limits for the API response.



## Component Sync

Passing data from the JS sandbox back to Python  
Streamlit state required careful URL parameter handling.



# Future Improvements

---

1

## Auth

User accounts to save long-term progress history.

2

## Languages

Add support for Rust, TypeScript, and SQL.

3

## Leaderboard

Global ranking system for competitive typing.

4

## IDE Plugin

Port logic to a VS Code extension.

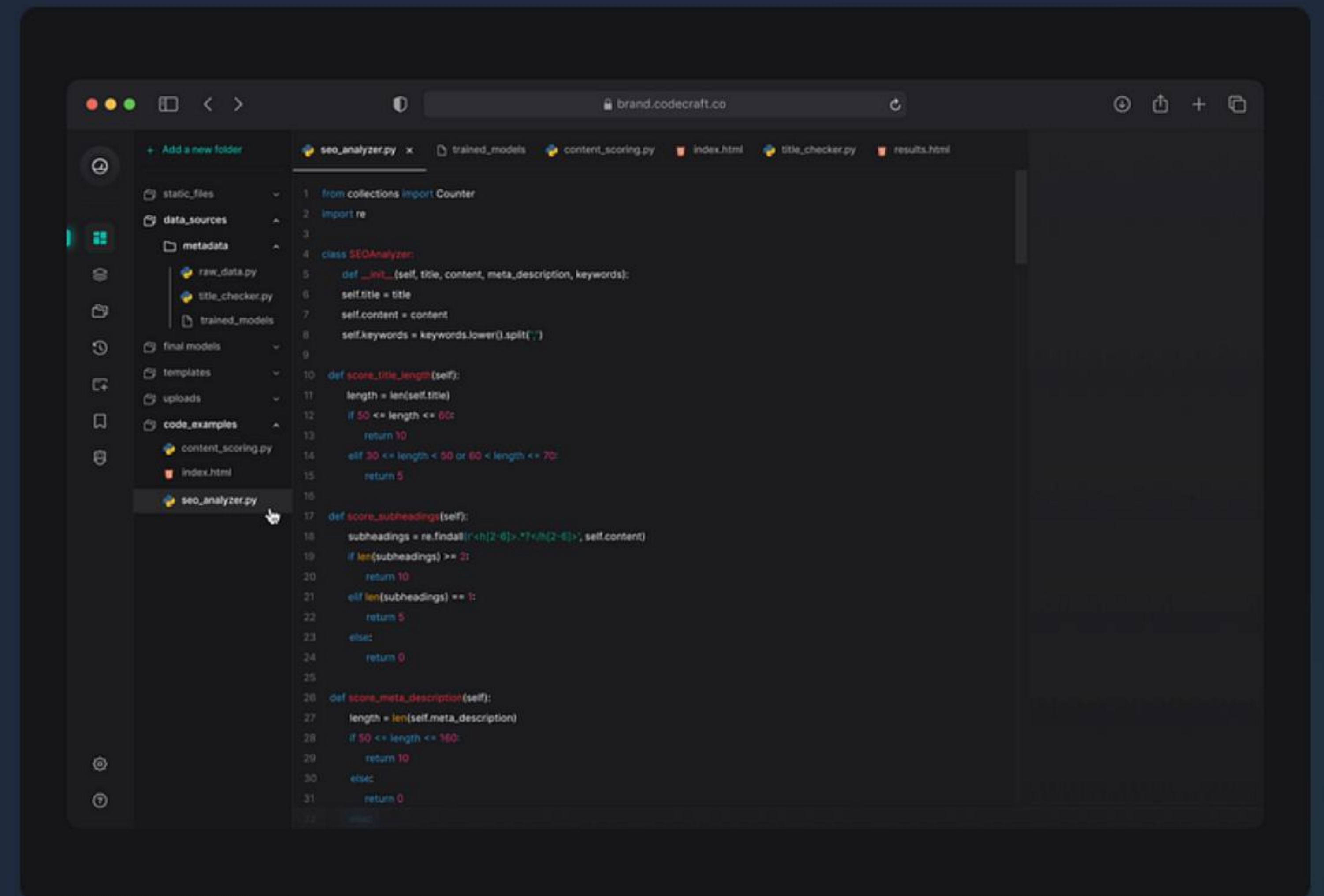


# Project Summary

## CodeType Trainer

A full-stack application demonstrating the power of integrating Generative AI with traditional web frontend logic.

- **</> Functional:** Real-time syntax practice.
- **</> Technical:** Advanced usage of Streamlit Components.
- **</> AI-Driven:** Custom Fine-Tuned Model.





# Questions?

Thank you for your time.



/jonsteen



jon@example.com



# Image Sources

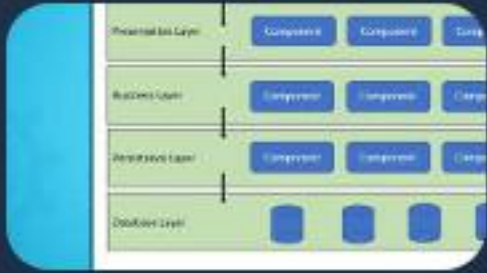
---



<https://www.mecexis.com/media/images/DDSC000034.2e16d0ba.fill-1800x1200.jpg>

Source: [www.mecexis.com](https://www.mecexis.com)

---



<https://media2.dev.to/dynamic/image/width=1080,height=1080,fit=cover,gravity=auto,format=auto/https%3A%2F%2Fdev-to-uploads.s3.amazonaws.com%2Fuploads%2Farticles%2F7u6nto6ga64yf8xyxt4z.jpg>

Source: [dev.to](https://dev.to)

---



<https://images.presentationgo.com/2025/04/ai-neural-network-brain.jpg>

Source: [www.presentationgo.com](https://www.presentationgo.com)

---



<https://cdn.dribbble.com/userupload/15895749/file/original-496e77ad32075f317033850ba82b14cc.png?resize=752x&vertical=center>

Source: [dribbble.com](https://dribbble.com)

---



<https://cdn.dribbble.com/userupload/43507872/file/original-df7180b18e97f8487d50bd65cba0a013.png>

Source: [dribbble.com](https://dribbble.com)